

CS 4240: Compilers & Interpreters

8/18/2025

Homework 1: Intermediate Representations & Optimization

Professor Qirun Zhang

Due: 9/10/2025 11:59pm

- We recommend you type your solutions using Overleaf or any other software. Pictures of hand written solutions will be accepted.
- All homework must be done independently. You may post clarification questions on Piazza, but be sure to make your post instructor-only if it includes part of your solution.

1 Intermediate Representations

(15 points) Show how the following source code fragment can be translated to Tiger-IR. Also, explain your answer in a few sentences. Assume standard rules of operator precedence/associativity as in C & Java, and pay attention to the correctness of the branch conditions that you introduce. As in the class and worksheet examples, you will need to introduce temporary variables. In this problem, please do a “straightforward” translation and try to refrain from doing control-flow changes or code optimizations.

```
i = 0;
while (i < 100) {
    if (c[i] == 0) {
        a[i] = (x + y + 2) * (a[i] - b[i]);
    } else {
        d[i] = c[i] * (x + y);
    }
    i = i + 1
}
```

```

#start_function
void prob_1(int x, int y, int[] a, int[] b, int[] c, int[] d):
int-list: i, t0, t1, t2, t3          # INSERT OTHER TEMPS HERE
float-list:                          # NO FLOATS

    assign, i, 0                      # i = 0

while_start:
    brgeq, while_end, i, 100         # if (i >= 100) GOTO while_end

    array_load, t0, c, i              # t0 = c[0]
    brneq, not_if, t0, 0              # if(t0 = c[0] != 0) GOTO not_if
    add, t1, x, y                     # t1 = x + y
    add, t1, t1, 2                     # t1 + 2 = x + y + 2
    array_load, t2, a, i
    array_load, t3, b, i
    sub, t2, t2, t3                    # a[i] - b[i] // t3 USABLE
    mult, t1, t1, t2                  # t1 = (x + y + 2) * (x + y) // t2 USABLE
    array_store, t1, a, i              # a[i] = t1
    goto, end_if

not_if:
    add, t1, x, y                     # t1 = x + y
    mult, t0, t0, t1                  # t0 = c[i] * (x + y)
    array_store, t0, d, i              # d[i] = t0

end_if:
    add, i, i, 1                      # i = i + 1
    goto, while_start

while_end:
#end_function

while_end:                            # end of while loop
#end_function

```

Note: for Problem 1, I assume the code fragment were a function named *prob_1* which did not return anything. All variables and arrays not declared in the code fragment were assumed to be inputs to the function.

2 Control Flow Graphs, Dominators, Reaching Definitions

Consider the Tiger-IR fragment shown below. Please refer to the Tiger-IR Reference Manual in the “General Resources” section in Piazza. Note that **puti** prints to standard output, and **geti** reads from standard input.

```
1. callr, t0, geti
2. assign, t1, 0
3. assign, t2, 0
4. assign, t3, 0
5. call, puti, t1
6. lstart:
7. brgeq, lexit, 1, t2
8. callr, t2, geti
9. mult, t0, t0, 5
10. goto, lstart
11. lexit:
12. call, puti, t2
13. assign, t3, 0
14. brgeq, else1, 0, t0
15. assign, t3, 0
16. else1:
```

- (a.) (5 points) Show a control flow graph for this IR code, using maximal basic blocks. Note that it's OK for a basic block to include only a label.

Answer has been included on the image below for (c.) (near the middle of the page).

- (b.) (5 points) Show the dominator tree for the CFG that you produced in part a).

Answer has been included on the image below for (c.) (near the top right of the page). Please note the simplifying assignments have been included in (a.) (i.e. E corresponds to $assign, t3, 0$).

- (c.) (20 points) For each def in the IR program, list all the uses that it reaches. Show the output of performing dead/useless code elimination, using the improved reaching-definitions worklist algorithm discussed in class (that still treats all branch instructions as critical instructions). Explain your answer in a few sentences, focusing on the role of critical instructions and worklist processing.

1 `Callr, t0, geti` # call geti
 2 `assign, t1, 0`
 3 `assign, t2, 0` = `Legders`
 4 `assign, t3, 0` `up first inst`
 5 `call, puti, t1` `up target of goto/branch`
 6 `lstart:` = `def`
 7 `brseq, lexit, 1, t2`
 8 `Callr, t2, geti`
 9 `mult, t0, t0, 5`
 10 `goto, lstart`
 11 `lexit:`
 12 `call, puti, t2`
 13 `assign, t3, 0`
 14 `brseq, elc1, 0, t0`
 15 `assign, t3, 0`
 16 `else1:`

a)

```

graph TD
    A["Callr, t0, geti  
assign, t1, 0  
assign, t2, 0  
assign, t3, 0  
call, puti, t1"] --> B["lstart:  
brseq, lexit, 1, t2  
(1 ≥ t2)"]
    B -- TRUE --> D["lexit:  
call, puti, t2  
assign, t3, 0  
brseq, elc1, 0, t0  
(0 ≥ t0)"]
    B -- FALSE --> G["Callr, t2, geti  
mult, t0, t0, 5  
goto, lstart"]
    D -- TRUE --> F["else1:  
assign, t3, 0"]
    D -- FALSE --> E["assign, t3, 0"]
    G --> B
  
```

b)

Block	Dom	IDom
A	A	—
B	AB	A
C	ABC	B
D	ABD	B
E	ABDE	D
F	ABDF	D

★ Note: I assume the following correlations to the pink marks have been made in (a).

c) defs/uses in the IR program (using CFG):

def	def line	use(s)
t0	1	9, 14
t1	2	5
t2	3	7, 12
t3	4	—
t2	8	7, 12
t0	9	9, 14
t3	13	—
t3	15	—

def	def line	use(s)
t0	1/9	9, 14
t1	2	5
t2	3/8	7, 12
t3	4/13/15	—

1 `Callr, t0, geti`
 2 `assign, t1, 0`
 3 `assign, t2, 0`
 4 `call, puti, t1`
 5 `lstart:`
 6 `brseq, lexit, 1, t2`
 7 `Callr, t2, geti`
 8 `mult, t0, t0, 5`
 9 `goto, lstart`
 10 `lexit:`
 11 `call, puti, t2`
 12 `brseq, elc1, 0, t0`
 13 `else1:`

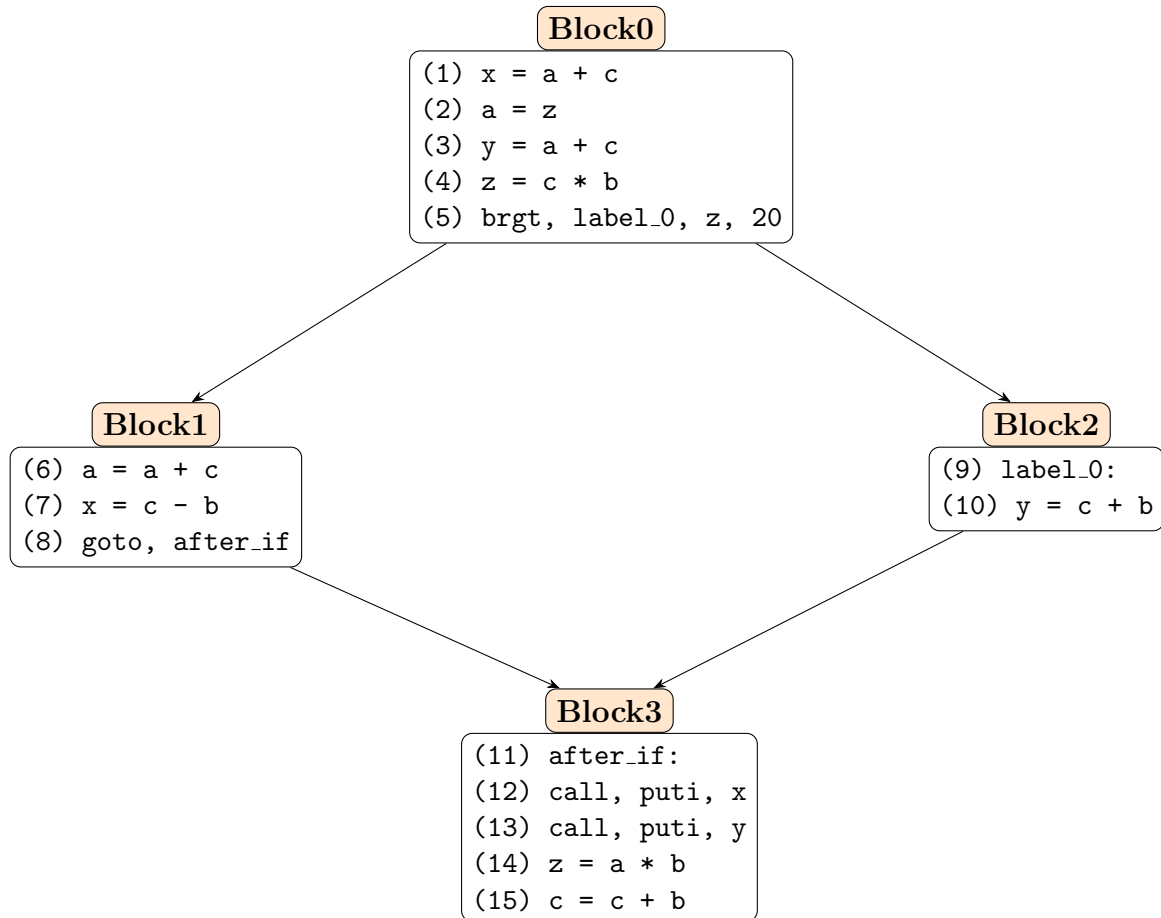
As we examine the code snippet, we mark critical instructions (branch, goto, I/O) and add them to the WorkList. We pop instructions from the worklist, tracing them backwards to identify their antecedents and assignments. We also mark these since they are necessary for critical operations. Instructions not marked are removed (such as the t3 assignments).

- (d.) (10 points) Discuss in a few sentences if there are additional dead instructions that were not identified by the improved reaching-definitions worklist algorithm in part c), and what techniques might be needed to identify them as such. Note that you do not need to consider removing labels (but it's OK if you remove redundant labels).

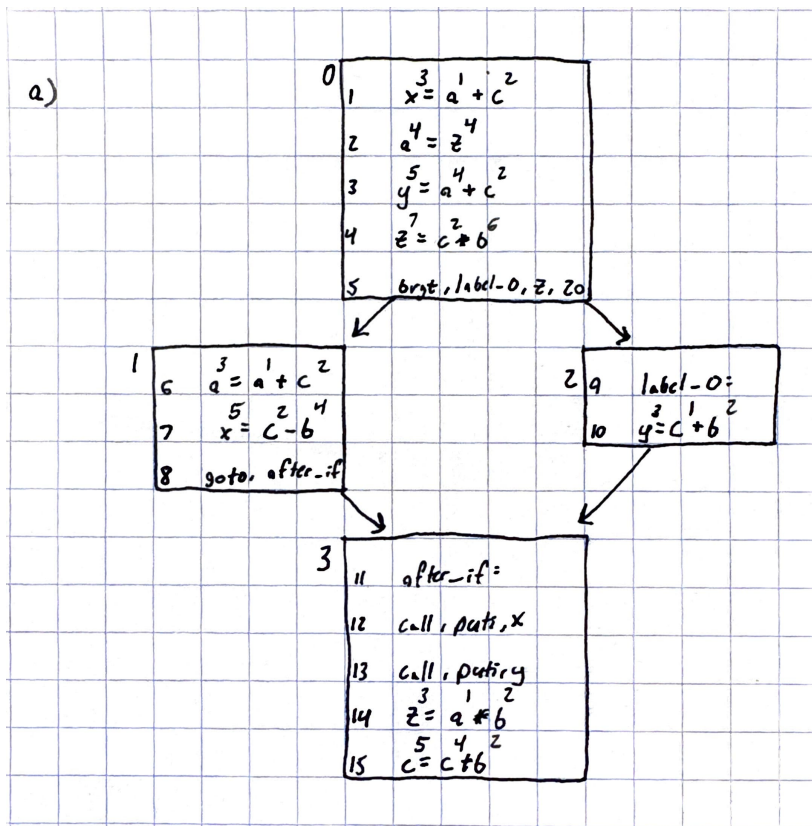
There are additional dead instructions that are not identify by the method used above. For example, we assign $t2 = 0$ on line 3, then on line 7, we always branch since the branch condition is always true. The loop body is not reachable in this case. To identify these, we may utilize constant propagation.

3 Value Numbering & Redundancy Elimination

Consider the following CFG with the IR code shown in each basic block. Please answer the following questions. Show value numbers assigned to variables and expressions. Please use positive integers starting from 1. You don't need to use any real hash functions. Instead, just utilize the next available positive integer.

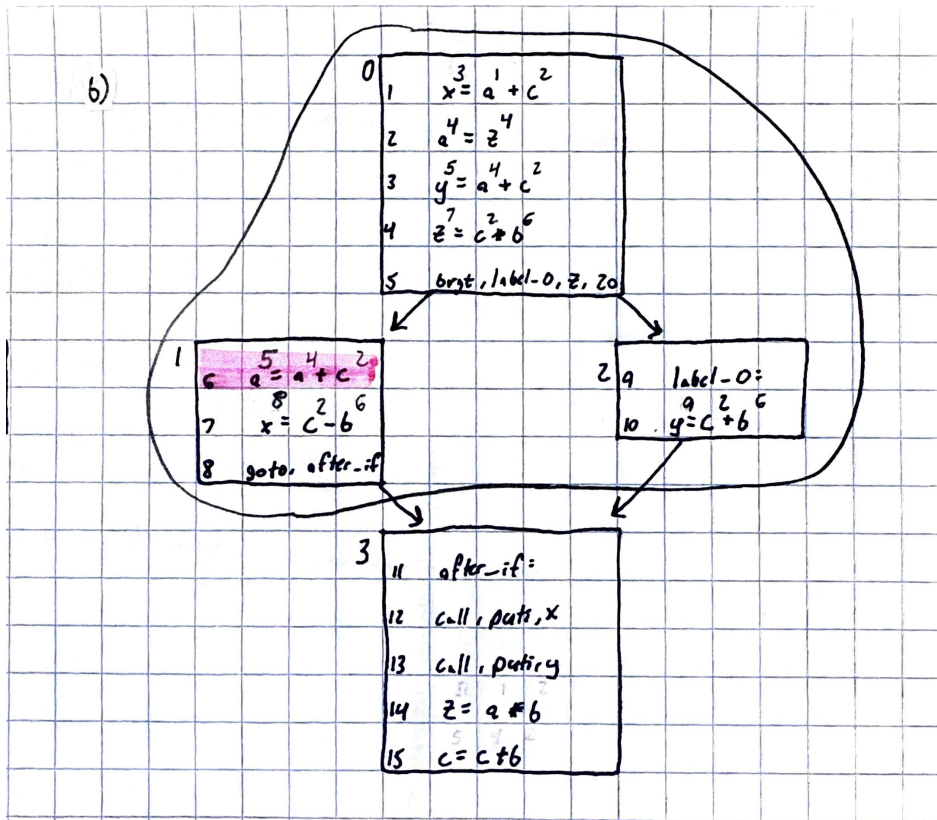


- (a.) (15 points) Show the redundant expression computation detected by the Local Value Numbering (LVN) technique only (limited to basic block by basic block analysis). Show the transformed code generated after the redundancy is removed. Introduce additional temporaries to hold the sub-expression results if necessary.



Within each Basic Block, LVN does not find any redundant instructions. For each block, the value numbers are shown as superscripts above. The transformed code is shown in the output above and is identical to the original code. No temporaries are introduced.

- (b.) (15 points) Repeat the above step for Superlocal Value Numbering (SVN) working on the scope of Extended Basic Block (EBB). How many additional redundancies are detected and eliminated by SVN compared to LVN? Explain your answer.

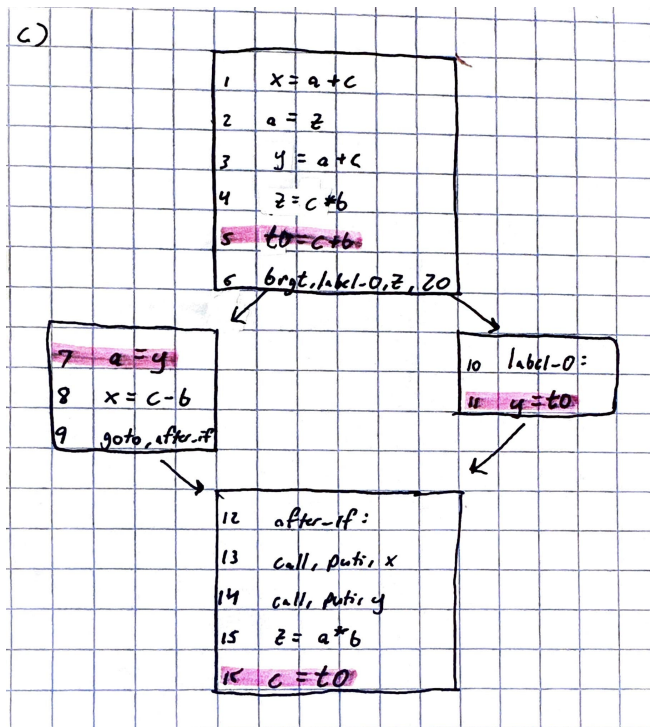
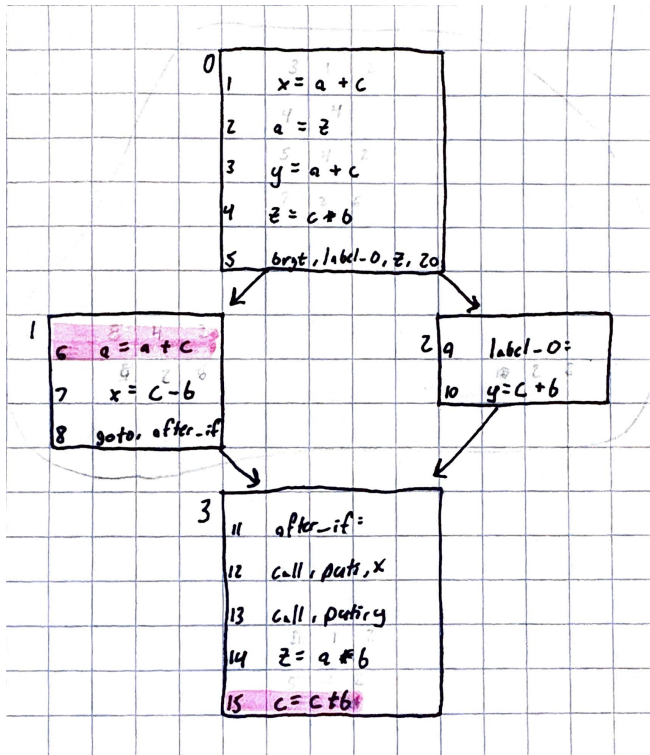


SVN detects one additional redundancy (as opposed to 0) compared to LVN. In the image above, the circled region is what is analyzed by EBB. Since Block 3 is a degenerate EBB and we know Block 3 alone will not produce a redundancy (as per LVN), we will not include this in the analysis.

The redundancy is found on line 6 (highlighted), as evidenced by the value numbers on line 3 and line 6 being identical. The transformed code would establish $a = y$ on line 6 instead of computing $a + c$ again.

Since we reuse an existing variable carrying the same value, there is not a need for a temporary variable.

- (c.) (15 points) Detect and eliminate the redundant expressions that are left out (if any) by SVN. Again, introduce necessary temporaries to hold the sub-expression results and show the transformed code. You don't need to consider dead code elimination or any other optimizations: just consider redundant expressions. Explain your answer.



In the first image, the highlighted instructions are redundant. For the instruction in Block 1, this was identified with SVN. For the instruction in Block 3, this is partially redundant since it follows the same instruction as line 10. Both line 10 and line 15 compute the assignment $c + b$.

To eliminate redundancy, we introduce a temporary variable in Block 1 (before the differing

paths) and assign it the value of $c + b$.

In the second image, all highlighted instructions are part of the transformed code.